UTM
UNIVERSITI TEKNOLOGI MALAYSIA
Faculty of Artificial Intelligence

TOPIC 4:

INTRODUCTION TO DATA WRANGLING WITH PANDAS (PYTHON)

DATA WRANGLING AND VISUALIZATION (SAIA2143)

Prepared by:
Ts. Dr. Norziha Megat Mohd Zainuddin
norziha.kl@utm.my

What is Data Analysis?



a. Process of inspecting, cleaning, and modeling data

b. Used to discover useful information and support decision-making

Terms

Term	Type	Description
Python 	A general-purpose **programming language**.	Web development, automation, data analysis, AI/ML, scripting, etc.
Pandas 	A **third-party library** built for Python, specialized for data manipulation and analysis.	powerful data structures like: Series – 1D labeled array DataFrame – 2D table (like an Excel sheet or SQL table)
Python Pandas	Usage term	Informal term for using Pandas in Python

Why Python?

- Easy to learn and widely used.
- Rich ecosystem of libraries for data analysis and visualization.



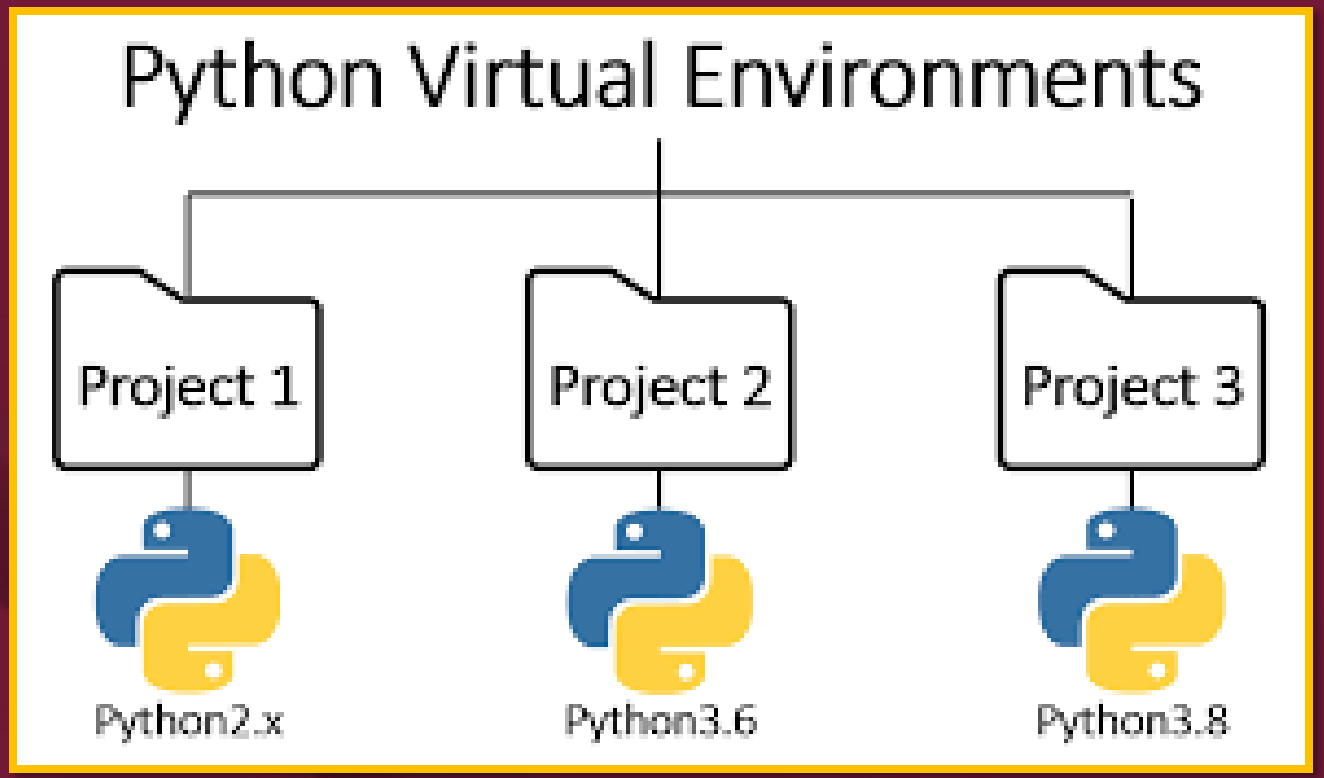
Class Activity: Python Libraries for Data Analysis



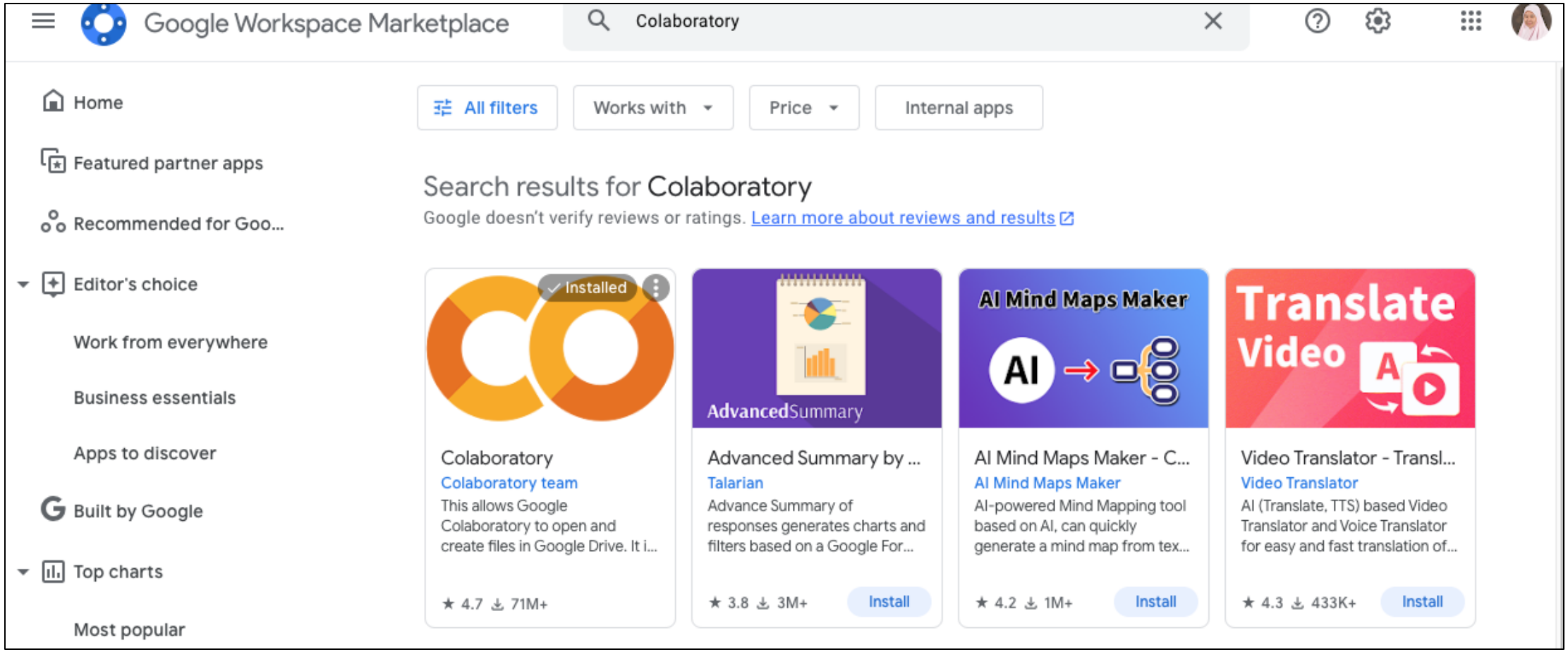
- Let's visit this website:

https://moonlighto2.medium.com/key-python-libraries-for-data-analysis-and-code-examples-f15c8a2349c1

Setting Up The Environment



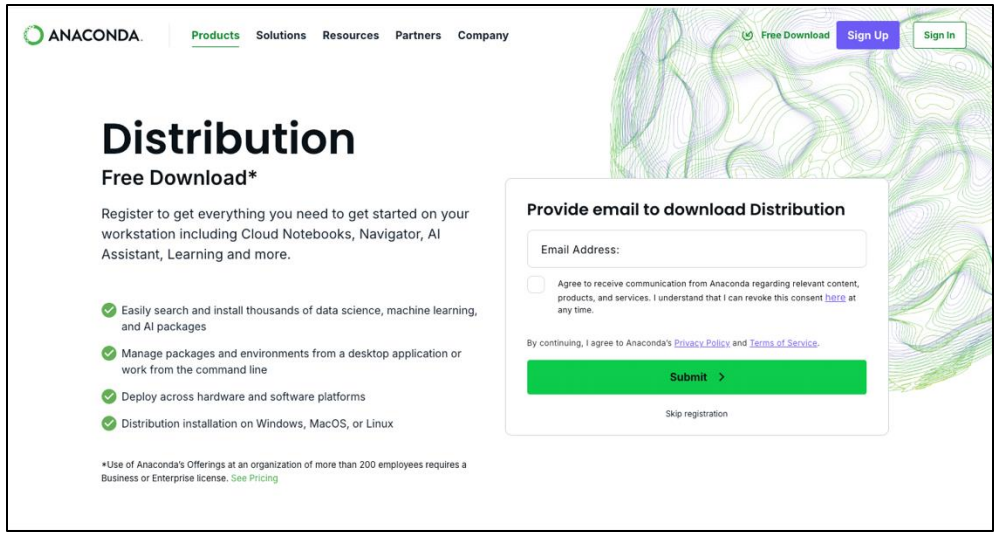
In Google Workspace Marketplace



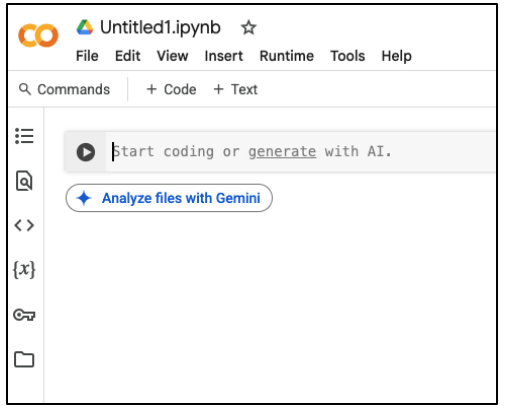
Anaconda vs Google Colab

Option 1: Install Anaconda to active Jupyter Notebook

- https://www.anaconda.com/products/distribution



Option 2: Use Google Colab (No Installation Needed)



- ✅ **Steps:**

1. Go to: https://colab.research.google.com
2. Sign in with your **Google account**.
3. Click **"New Notebook"** to start coding.

Google Colab

How to run Pandas with Python using Google Colab

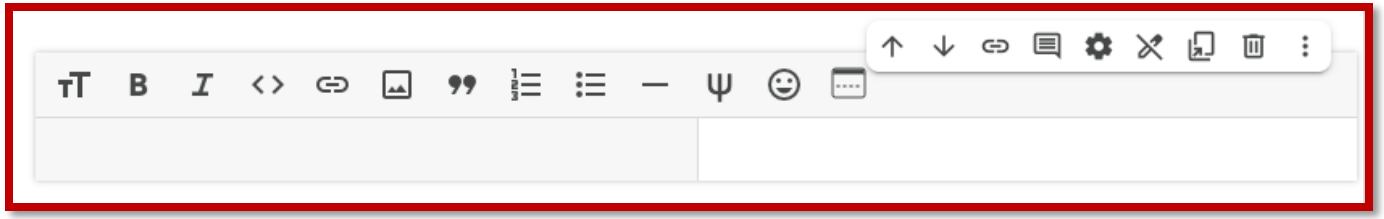


1. Type the text
print ("Hellow World")

2. Run button (press it) or Ctrl + Enter

3. Display the output

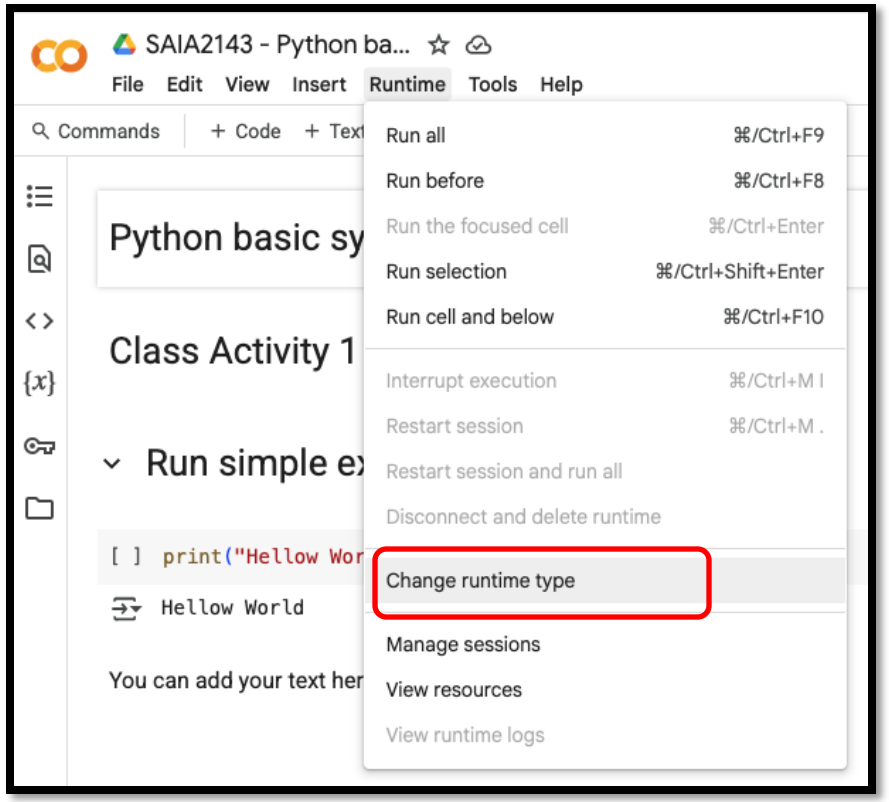
+ Text / Markdown cells



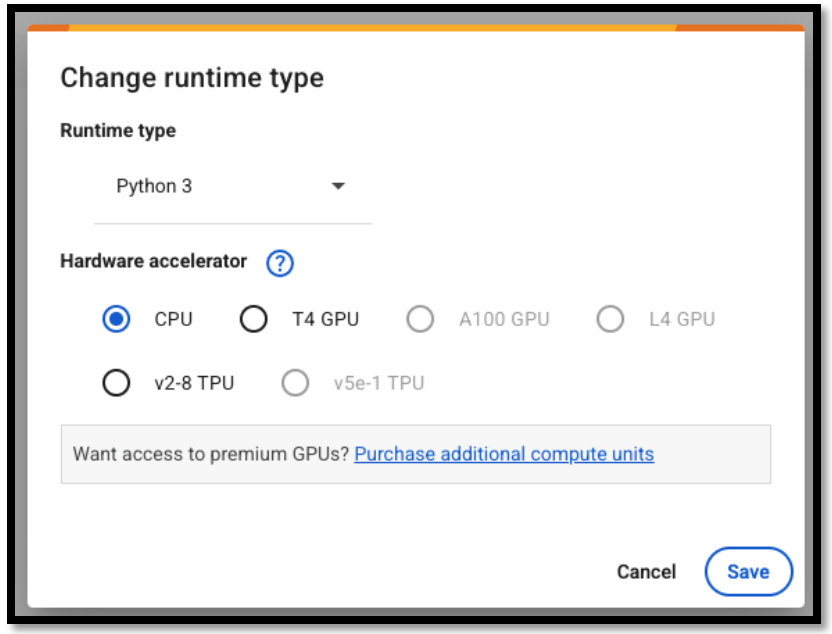
Function	Description
Add Explanations	Describe what your code does, explain outputs, or give context to your notebook
Format Text	Use Markdown to add bold, italics, bullet points, numbered lists, etc.
Add Headings/Subheadings	Organize your notebook with section titles using #, ##, etc.
Insert Images/Links	Embed images () or hyperlinks ([text](url))
Display Math Equations	Use LaTeX syntax to show formulas (e.g., $E = mc^2$)
Improve Navigation	Create a cleaner and more readable notebook for users and reviewers

Change Runtime in GC

Runtime -> Change runtime type



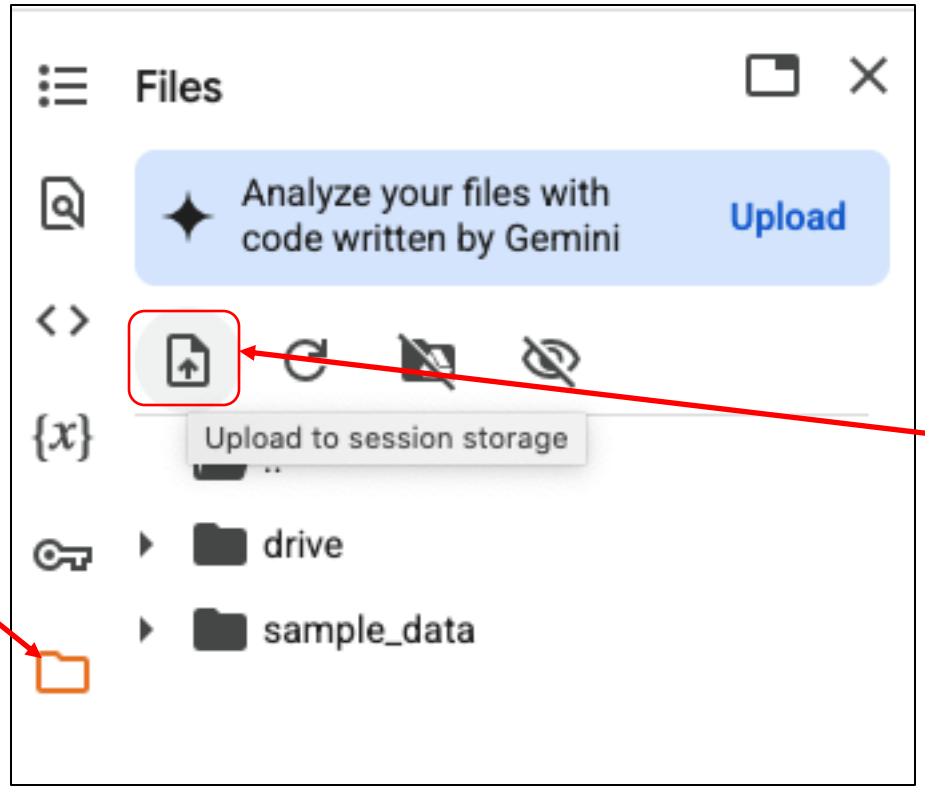
Choose CPU to run



T4 GPU- for powerful Graphics Processing Unit, example image and video processing

Upload Files to Google Colab

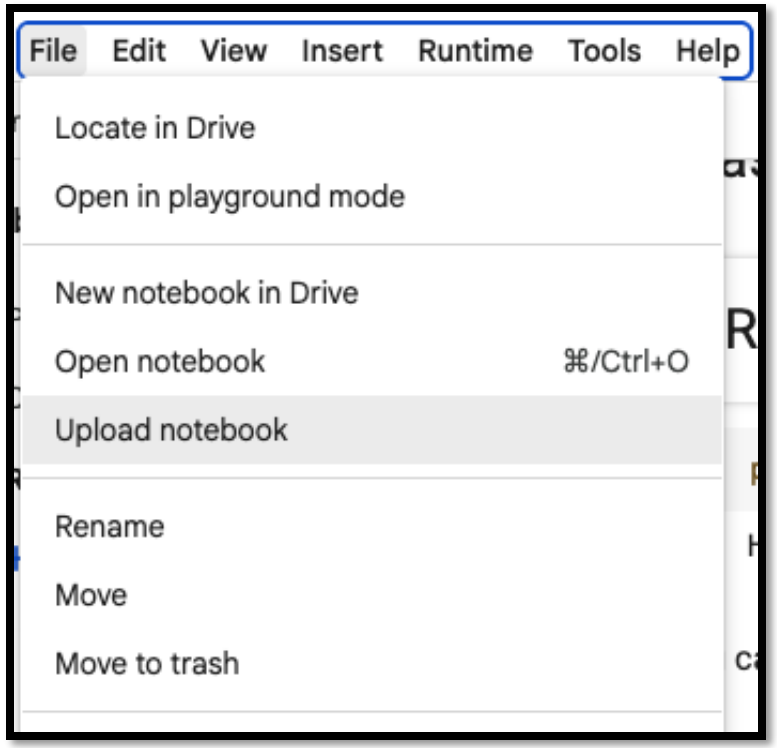
1. Files



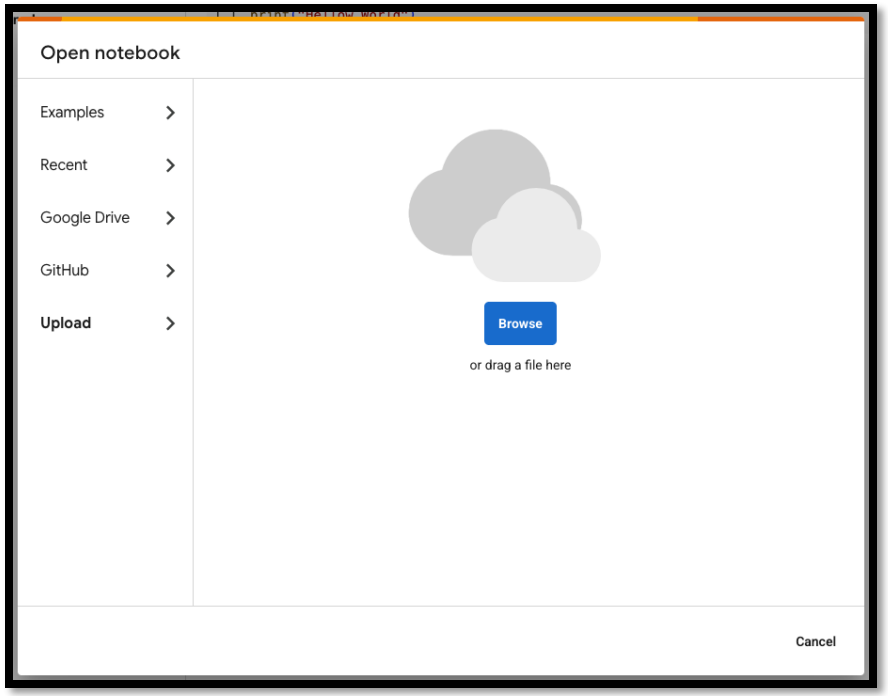
2. Click on upload file and choose file from your computer.

How to Upload and Open ipynb File

File-> Upload notebook



Browse or drag to this area





CLASS ACTIVITY

Instruction:

1. Please upload the file given in e-learning and upload to Google Colab

"Python_Basic_Syntax_Exercise.ipynb"

2. Try to run every code



Download File

Interactive Python Notebook; file format used by **Jupyter Notebooks**



Plain text for **Py**thon programming language.

Python Basic Syntax for Data Analysis



Python Basic Syntax for Data Analysis

1. Comments in Python

- Comments are not executed
- Used to explain code
- Syntax: # This is a comment

2. Printing Output

- Use **print()** to display text or variables
- print("Hello, World!")

Python Basic Syntax for Data Analysis

3. Variables and Data Types

- Python is dynamically typed

```
name = "Ali"          # String
age = 21              # Integer
height = 1.75         # Float
is_student = True     # Boolean
```

4. Indentation in Python

- Indentation defines code blocks

```
if age > 18:
    print("Adult")
```

Python Basic Syntax for Data Analysis

5. Type Checking

- Use **type()** to check data type

```
type(name)  # <class 'str'>
```

6. Operators

- **Arithmetic**: +, -, *, /
- **Comparison**: ==, !=, >, <, >=, <=

```
a = 5 + 3
x = 5 > 3
```

Python Basic Syntax for Data Analysis

7. Conditional Statements

- Control the flow with **if, elif, else**

```
if age < 18:
    print("Minor")
elif age == 18:
    print("Exactly 18")
else:
    print("Adult")
```

8. Loops

- For loop: **for i in range(5):**
- While loop: **while count < 5:**

```
for i in range(5):
    print(i)
```

Python Basic Syntax for Data Analysis

9. Functions

- Define reusable code blocks with **def**

```
def greet(name):
    print("Hello, " + name)

greet("Ali")
```

10. Built-in Functions

- len(): Length of an object
- type(): Data type
- int(), str(), float(): Type conversion
- range(): Sequence of numbers

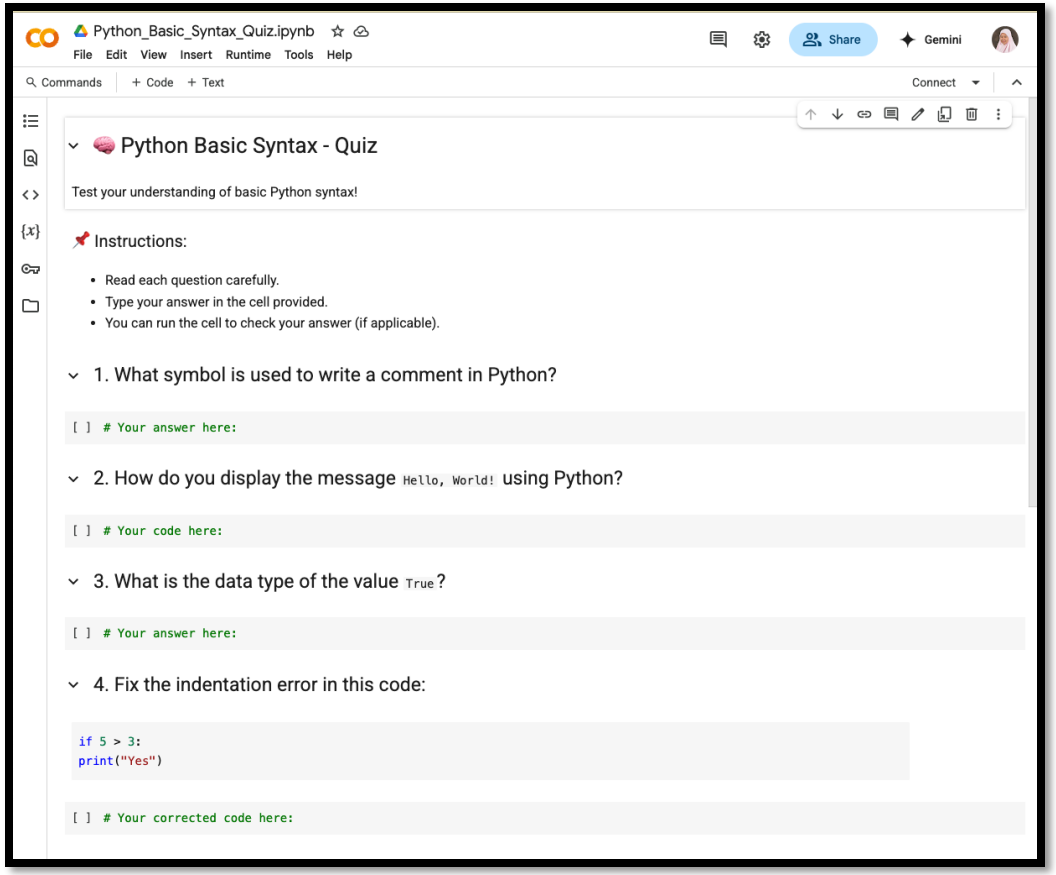
CLASS ACTIVITY

Instruction:

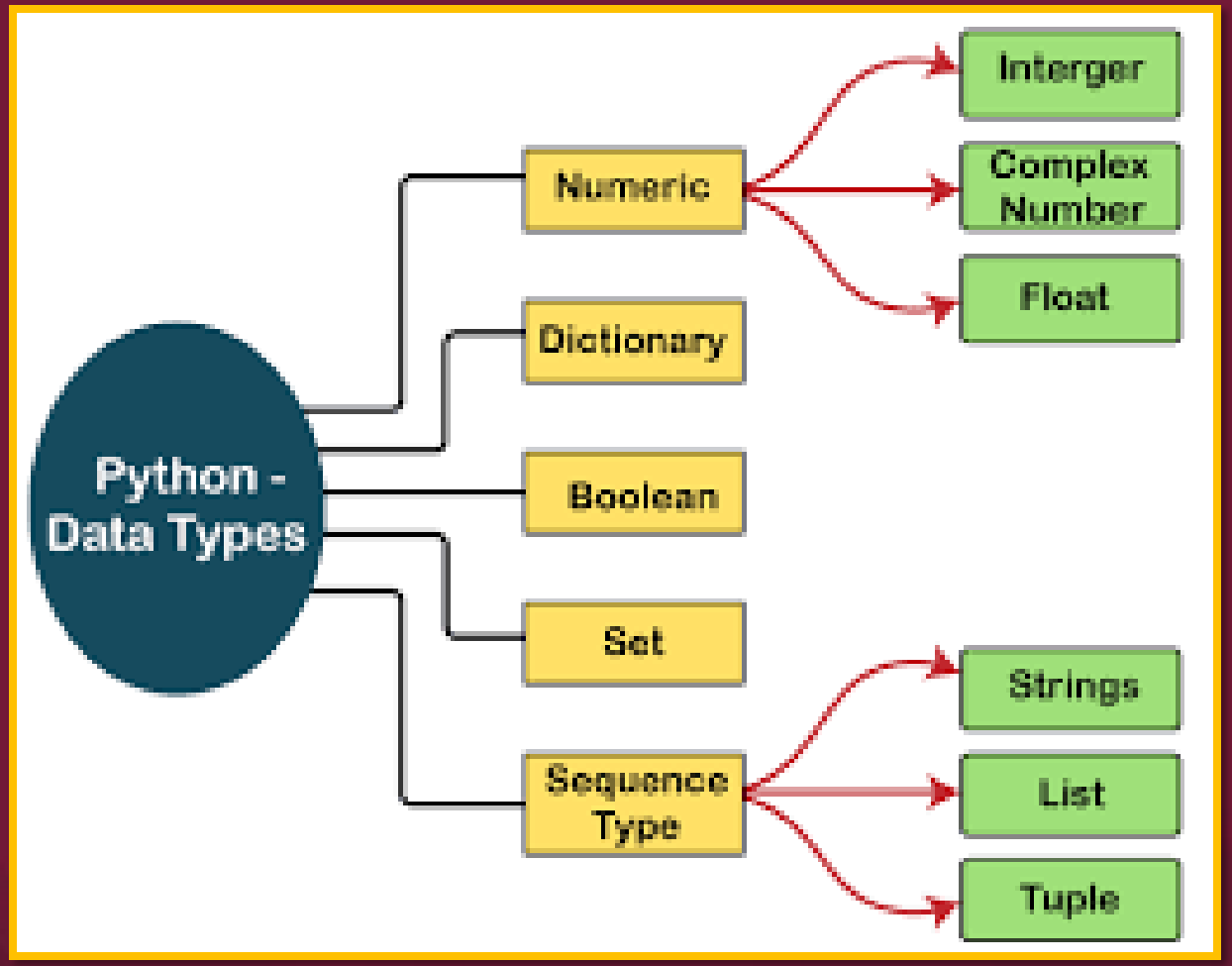
1. Please upload the file given in e-learning and upload to Google Colab

"Python_Basic_Syntax_Quiz.ipynb"

2. Try to answer every questions



Python Data Types



What are Data Types in Python?

- Data types tell Python what kind of value a variable holds
- Essential for performing operations correctly
- Python is dynamically typed (you don't declare types explicitly)

Examples:

```
x = 5          # int
y = 3.14       # float
z = "hello"    # str
```

Common Python Data Types

Type	Description	Example
int	Integer numbers	x = 10
float	Decimal numbers	y = 3.14
str	Text (string)	s = "Python"
bool	Boolean (True/False)	flag = True

Note: Use type() function to check a variable's type:

```
print(type(x))   # <class 'int'>
```

Data Structures in Python

Type	Description	Example
list	Ordered, mutable collection	[1, 2, 3]
tuple	Ordered, immutable collection	(1, 2, 3)
dict	Key-value pairs	{"name": "Ali"}
set	Unordered, unique items	{1, 2, 3}

Usage:

```
my_list = ["apple", "banana"]
my_dict = {"age": 21, "name": "Lina"}
```

Converting Between Data Types

- Python allows conversion using built-in functions:

```
x = int("5")          # str to int
y = float("3.14")     # str to float
z = str(100)          # int to str
```

Use Cases:

- Reading from user input (always returns str)
- Processing numeric values

Key Takeaways on Python Data Types

- Know your data: it affects how code behaves
- Use type() to inspect values
- Use int(), str(), etc., for conversions
- Explore more types: complex, bytes, NoneType

Challenge:

```
# Try guessing the types
print(type([1, 2, 3]))
print(type((1,)))
print(type({"a": 1}))
```



CLASS ACTIVITY

Part A: Match the Type to Description

Write the correct letter next to each number.

Type	Description	Your Answer
1. int	A. Text data	
2. str	B. Collection of key-value pairs	
3. bool	C. Whole numbers	
4. list	D. Logical value: True or False	
5. dict	E. Ordered, changeable list	



CLASS ACTIVITY

Part B: Match the Value to the Correct Type

Write the correct number next to each letter.

Value	Type	Your Answer
A. "hello"	1. bool	
B. True	2. float	
C. 3.14	3. list	
D. [1, 2, 3]	4. str	
E. {}	5. dict	



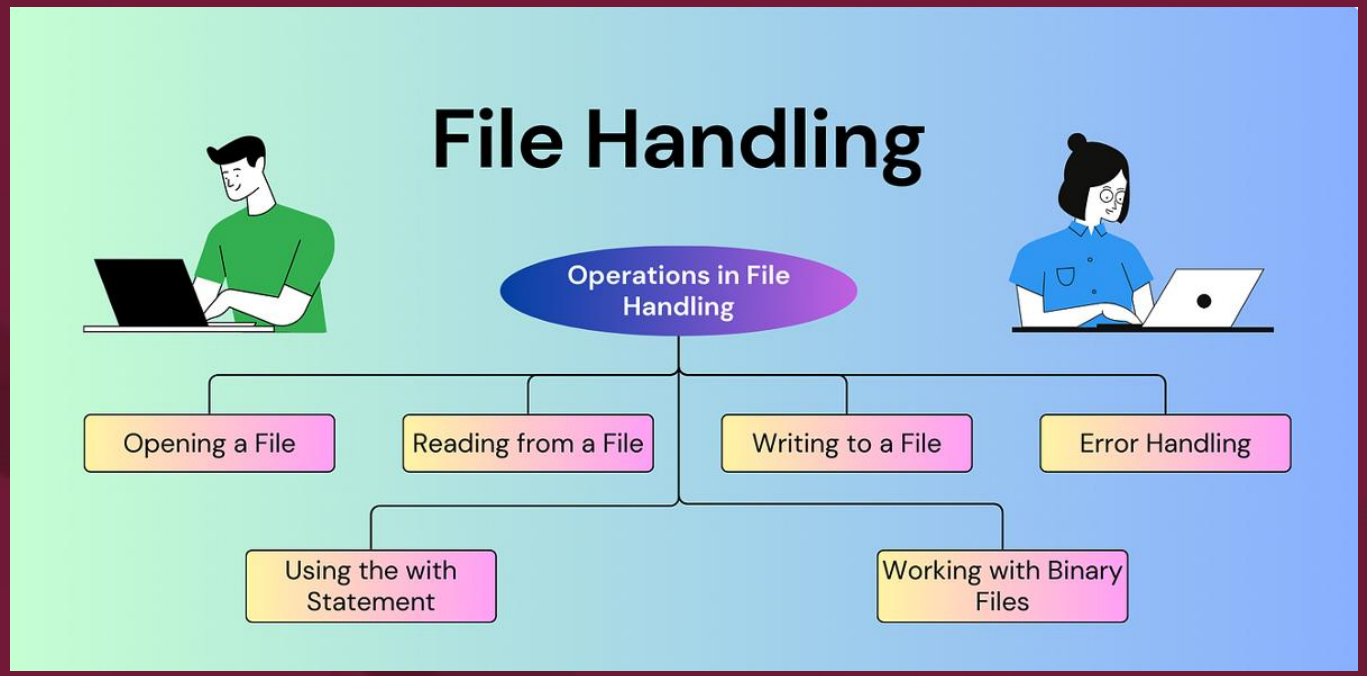
CLASS ACTIVITY

Part C: Match the Code to the Output Type

Write the correct letter next to each number.

Code	Output Type	Your Answer
1. type(5)	A. <class 'str'>	
2. type({"name": "Lina"})	B. <class 'bool'>	
3. type("5")	C. <class 'int'>	
4. type(True)	D. <class 'dict'>	
5. type([1, 2, 3])	E. <class 'list'>	

Functions and File Handling in Python



Functions

A reusable block of code that performs a specific task.

Why use functions?

- Reduce repetition
- Improve readability
- Make code modular and organized

Example:

```
def greet():
    print("Hello, world!")

greet() #Call the function
```

Function with Parameters and Return

Parameters: Allow data to be passed into a function **Return:** Sends a value back to the caller

Example:

```
def add(a, b):
    return a + b

result = add(5, 3)
print(result)  # Output: 8
```

File Handling Basics

Opening Files:

```
file = open('data.txt', 'r') # 'r' for read mode
```

Common Modes:

- 'r': Read
- 'w': Write (overwrite)
- 'a': Append
- 'r+': Read and write

Always close files!

```
file.close()
```

Reading from Text Files

Example:

```
with open('data.txt', 'r') as file:
    content = file.read()
    print(content)
```

Benefits of with statement:

- Automatically closes file
- Cleaner and safer

Writing to Text Files

Example:

```
with open('output.txt', 'w') as file:
    file.write("This is a test.")
```

- Overwrites if file exists
- Creates file if it doesn't exist

Working with CSV Files

Use the csv module:

```
import csv

with open('data.csv', 'r') as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

- Each row is a list of strings

Writing to CSV Files

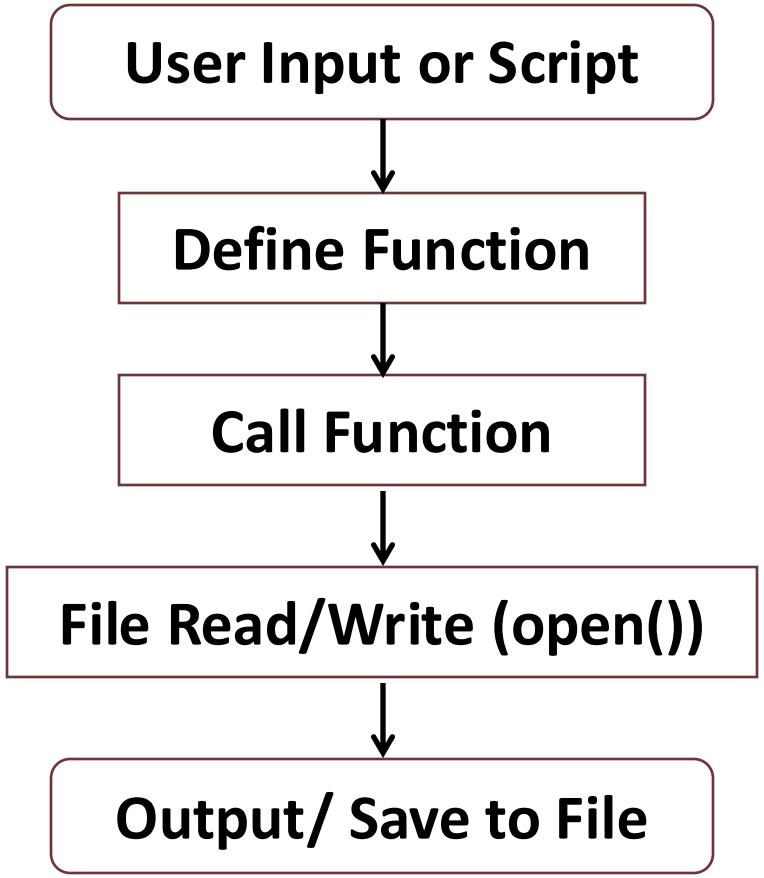
Example:

```
import csv

data = [["Name", "Age"], ["Ali", 23], ["Lina", 21]]

with open('students.csv', 'w', newline='') as file:
    writer = csv.writer(file)
    writer.writerows(data)
```

Function & File Handling Flow





CLASS ACTIVITY

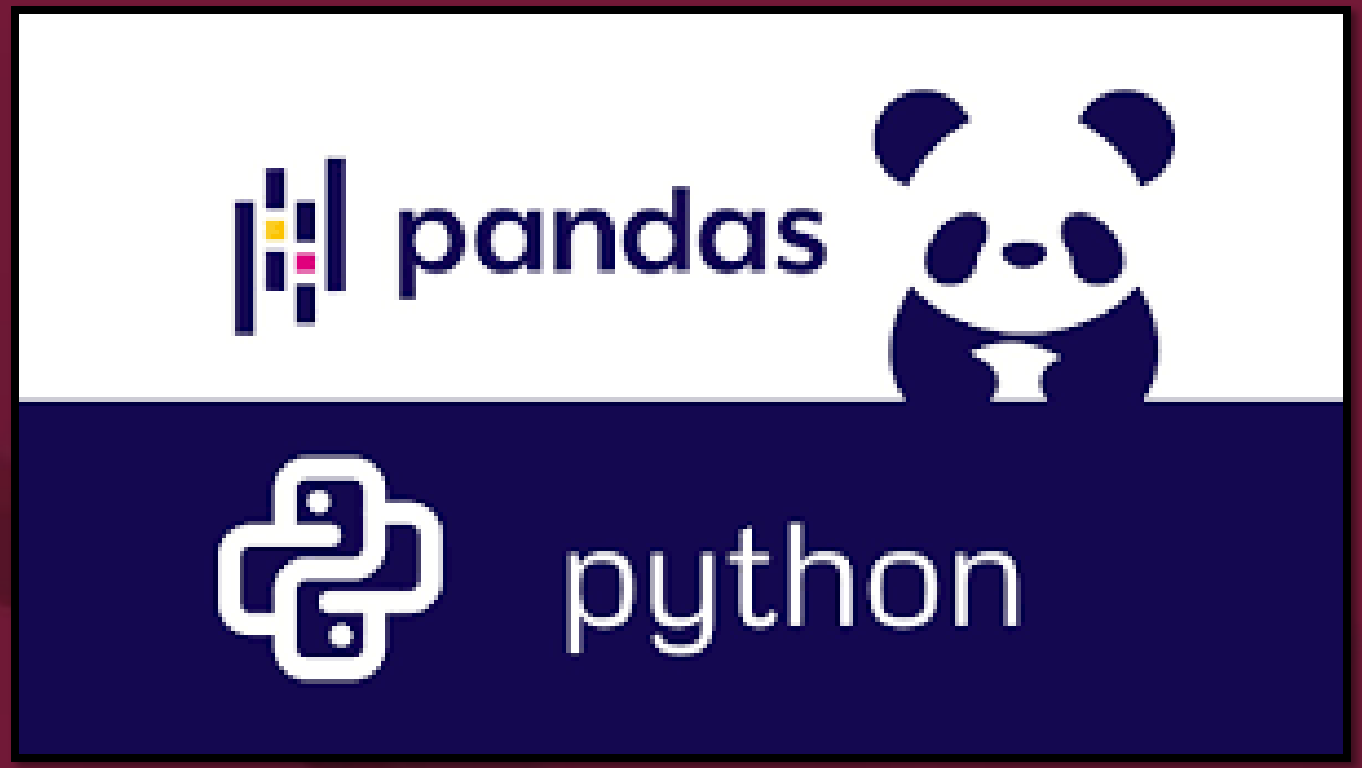
Mini Python Scripting

Instruction:

1. Define a function in Google Colab called `multiply_by_two(n)` that returns `n * 2`.
2. Write a function that takes a filename and returns the number of lines in it.
3. Create a CSV file with student data (Name, Grade), then read and display it.

Bonus: Modify the student data file to add a new student record.

Pandas Essentials



Introduction to Pandas

- **Pandas** is a powerful **open-source data analysis** and **manipulation library** in Python.

- It's especially designed to make **working with structured data (like tables)** easy and efficient.



Loading Libraries

📌 *Example:*

i. Import the package as pd

```
import pandas as pd
```

ii. We use the pd.read_csv function to read the dataframe [df] file as **pandas.DataFrame** object

```
df = pd.read_csv('sales_data.csv')
```

iii. To print the dataframe

```
print(df.head())
```

```
#The following libraries are used, so please load them beforehand
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Pandas Data Structure

Feature	Series	DataFrame
Dimensions	1D	2D
Structure	Single column	Table (rows/index + columns)
Data Type Support	Single data type or mixed	Each column can have its own type
Indexed?	Yes	Yes (rows/index and columns)
Example Shape	(3,)	(3,2)

Series (1 dimension)

		ID
Row 1	**0**	100
Row 2	**1**	101
Row 3	**2**	102
Row 4	**3**	103
Row 5	**4**	104

Index **Values**

DataFrame (2 dimension)

	Column 1	Column 2	Column 3	Column 4
	Product	**Sales**	**Price**	**In_Stock**
0	Apples	100	1.2	TRUE
1	Oranges	200	0.8	TRUE
2	Bananas	300	0.5	FALSE
3	Grapes	150	2	TRUE

Example of Pandas Series

Coding:

```
import pandas as pd

data = [10, 20, 30]
series = pd.Series(data)

print(series)
```

Output:

Class Activity: Please try in



Example of Pandas DataFrame

Coding:

```
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35]
}

df = pd.DataFrame(data)
print(df)
```

Output:

Class Activity: Please try in





CLASS ACTIVITY

Series and DataFrame

Instruction:

1. Import the pandas library.
2. Create a list of fruit names.
3. Create a custom index from A to J.
4. Use pd.Series() to build your Series.
5. Name your Series "Fruits" using the name= parameter.
6. Print the Series.

Example Output

```
A         Apple
B        Banana
C        Cherry
D          Date
E    Elderberry
F           Fig
G         Grape
H     Honeydew
I     Indian Fig
J       Jackfruit
Name: Fruits, dtype: object
```


www.fai.utm.my

SAMPLE ANSWER

```
import pandas as pd


# List of 10 fruits starting with A to J
fruit_names = [
    'Apple',        # A
    'Banana',       # B
    'Cherry',       # C
    'Date',         # D
    'Elderberry',   # E
    'Fig',          # F
    'Grape',        # G
    'Honeydew',     # H
    'Indian Fig',   # I
    'Jackfruit'     # J
]

# Custom index A to J
index_labels = list('ABCDEFGHIJ')

# Create the Series
fruits_series =
pd.Series(fruit_names,
index=index_labels, name='Fruits')

# Print the result
print(fruits_series)
```



CLASS ACTIVITY

Series and DataFrame

Instruction: Create a Pandas DataFrame with information about 10 fruits (A–J):

The DataFrame should have the following **4 columns**:

a. Fruit – the name of the fruit (use your previous Series)
b. Category – set all to "Fresh"
c. Price – enter the price per unit (you choose the values)
d. In_Stock – enter True or False to indicate availability

Use the **same A–J index** as before.

Example Output

	Fruit	Category	Price	In_Stock
A	Apple	Fresh	1.50	True
B	Banana	Fresh	0.75	True
C	Cherry	Fresh	2.00	False
D	Date	Fresh	3.00	True
E	Elderberry	Fresh	4.50	False
F	Fig	Fresh	2.50	True
G	Grape	Fresh	1.80	True
H	Honeydew	Fresh	3.20	True
I	Indian Fig	Fresh	5.00	False
J	Jackfruit	Fresh	4.00	True

www.fai.utm.my

Basic Data Operations in Pandas

Data Preprocessing:
Handle missing values

Data Cleaning:
Remove duplicates and fix inconsistencies

Data Transformation:
Filter, sort and modify data

Loading and Viewing Data	Selecting, Slicing and Filtering Data	Upload file
Filtering Rows	Sorting and Adding Columns	Renaming and Dropping
Cleaning Data	Duplicates	Reshaping Data

a. Loading and Viewing Data

- **Load data** using:
  - `pd.read_csv('file.csv')`
  - `pd.read_excel('file.xlsx')`
  - `pd.read_json('file.json')`

- **View data**:
  - `df.head()` – First 5 rows
  - `df.tail()` – Last 5 rows
  - `df.info()` – Data types and non-null values
  - `df.describe()` – Statistical summary



CLASS ACTIVITY

- **Objective:** Learn how to load an Excel file using pandas and explore its content using basic data inspection functions.
- **Instructions:** According to the Excel file **student_data.xlsx** identify:
  1. Import the pandas library
  2. Load the Excel file into a DataFrame using `pd.read_excel()`
  3. Use the following methods to **explore** the dataset:
     a. `df.head()`
     b. `df.tail()`
     c. `df.info()`
     d. `df.describe()`

ANSWER

```python
import pandas as pd

# Load the Excel file
df = pd.read_excel("students_data.xlsx")

# View first few rows
print("First 5 rows:")
print(df.head())

# View last few rows
print("\nLast 5 rows:")
print(df.tail())

# Show summary info
print("\nData Info:")
print(df.info())

# Show statistical summary
print("\nDescriptive Statistics:")
print(df.describe())
```

CLASS ACTIVITY

Instructions: According to the Excel file **student_data.xlsx** identify:

1. How many rows and columns are there?
2. What are the data types of each column?
3. What is the average, max, and min score?
4. How many students received an A grade?
5. What is the most common age among the students?
6. Find and display the records of students who scored **above 90**.

Answer

1. How many rows and columns are there?

```python
print("Shape of the DataFrame:", df.shape)
# Output: (rows, columns)
```

2. What are the data types of each column?

```python
print("Data types of each column:")
print(df.dtypes)
```

3. What is the average, max, and min score?

```python
print("Average Score:", df['Score'].mean())
print("Maximum Score:", df['Score'].max())
print("Minimum Score:", df['Score'].min())
```

4. How many students received an A grade?

```python
count_a = df[df['Grade'] == 'A'].shape[0]
print("Number of students with A grade:", count_a)
```

5. What is the most common age among the students?

```python
print("Most common age:", df['Age'].mode()[0])
```

```python
print(df[df['Score'] > 90])
```

b. Selecting and Filtering Data

- **Selecting data**:
  - One column: `df['ColumnName']`
  - Multiple columns: `df[['Col1', 'Col2']]`
  - Rows by position: `df.iloc[0:5]`
  - Rows by label: `df.loc[0:5, ['Col1']]`

- **Filtering data**:
  - `df[df['Price'] > 50]`
  - `df[df['Category'] == 'Fresh']`

Slicing

- Slicing - create a new df by taking a subset of row or colums out of another dataframe. (Vertical or horizontal)
- **Based on index position** (like using a slice of a list).
- Typically uses `.iloc[]` or `.loc[]` with **row positions** or **index labels**.
- Best when you want a **range of rows** by number.



Example:

```
df.iloc[1:4]  # Get rows from index position 1 to 3
df.loc[1:3]  # Get rows with index labels 1 to 3 (inclusive)
```

Different between Selecting and Slicing

Feature	Selecting	Slicing
Purpose	Pick specific columns or values	Get a range of rows (and sometimes columns)
Methods	`df['col'], df[['col1', 'col2']]`	`df[0:5], df.iloc[], df.loc[]`
Focus on	Columns	Rows (and optionally columns)





c. Upload file

Jupyter Notebook

- Copy the path of the file name
- Mac OS use right button and press Alt

Google Colab

- Use this code to upload *popular_songs.csv* directly:

```
from google.colab import files
uploaded = files.upload()
```

- Once uploaded, you can read it like this:

```
import pandas as pd

df = pd.read_csv('popular_songs.csv')
print(df['Artist'])
```

```python
import pandas as pd

from google.colab import files
uploaded = files.upload()


df = pd.read_csv('popular_songs.csv')  # Use the name exactly as uploaded
print(df['Artist'])
```

```
Choose files popular_songs.csv
• popular_songs.csv(text/csv) - 746 bytes, last modified: 22/04/2025 - 100% done
Saving popular_songs.csv to popular_songs (1).csv
0                       The Weeknd
1                       Ed Sheeran
2     Mark Ronson ft. Bruno Mars
3                            Adele
4                            Adele
5    Luis Fonsi ft. Daddy Yankee
6                     Billie Eilish
7                         Lil Nas X
8                          Dua Lipa
9                       Ed Sheeran
Name: Artist, dtype: object
```

Example for Selecting and Slicing

Instruction: According to the file "popular_song.csv" identify:

1. Select One Column

```
print(df['Artist'])  # Select the 'Artist' column
```

2. Select Multiple Columns

```
print(df[['Title', 'Genre']])  # Select 'Title' and 'Genre'
```

3. Slicing First 3 Rows

```
print(df.iloc[0:3])  # Use iloc to slice first 3 rows (0,1,2)
```

4. Slicing Rows and Specific Columns using loc

```
print(df.loc[0:5, ['Title', 'Release Year']])
# Select rows 0 to 5, and columns 'Title' and 'Release Year'
```

Google Colab file:

Selecting_and_filtering.ipynb

www.fai.utm.my

```python
import pandas as pd

from google.colab import files
uploaded = files.upload()


df = pd.read_csv('popular_songs.csv')  # Use the name exactly as uploaded
print(df['Artist'])

print (df[['Title','Genre']]) # This line was moved from a separate cell to the current cell

print(df.iloc[0:3])

print(df.loc[0:5, ['Title','Release Year']])
```

```
Choose files popular_songs.csv
• popular_songs.csv(text/csv) - 746 bytes, last modified: 22/04/2025 - 100% done
Saving popular_songs.csv to popular_songs (2).csv
0                    The Weeknd
1                    Ed Sheeran
2     Mark Ronson ft. Bruno Mars
3                          Adele
4                          Adele
5    Luis Fonsi ft. Daddy Yankee
6                  Billie Eilish
7                      Lil Nas X
8                       Dua Lipa
9                    Ed Sheeran
Name: Artist, dtype: object
                Title        Genre
0      Blinding Lights    Synth-pop
1         Shape of You          Pop
2          Uptown Funk     Funk-pop
3  Rolling in the Deep         Soul
4     Someone Like You   Pop ballad
5            Despacito    Reggaeton
6              Bad Guy   Electropop
7        Old Town Road  Country rap
8           Levitating    Disco-pop
9              Perfect          Pop
              Title                     Artist          Album  Release Year \
0  Blinding Lights                  The Weeknd     After Hours          2019
1     Shape of You                  Ed Sheeran          Divide          2017
2      Uptown Funk  Mark Ronson ft. Bruno Mars  Uptown Special          2014

  Length      Genre Language  Chart Position
0   3:20  Synth-pop  English               1
1   3:53        Pop  English               1
2   4:30   Funk-pop  English               1
                Title  Release Year
0      Blinding Lights          2019
1         Shape of You          2017
2          Uptown Funk          2014
3  Rolling in the Deep          2010
4     Someone Like You          2011
5            Despacito          2017
```

wrap text that's too long to fit in one line or column when displayed.



CLASS ACTIVITY

Dataset: **popular_songs.xlsx**

1. Display the **Album names** only.
2. Show the first **3 songs** with their **Title** and **Chart Position**.
3. Use .loc[] to display the **first 4 songs** and only their **Artist** and **Length**.
4. Display all songs where the **Language** is English and show only **Title** and **Genre**.
5. Filter and display songs that were released **after 2015** and are in **English**.

ANSWER

🧠 Practice Questions with Answers

> 📝 Make sure you load the Excel data first:

```python
import pandas as pd
df = pd.read_excel("popular_songs.xlsx")
```

✅ 1. Display the Album names only

```python
print(df['Album'])
```

✅ 2. Show the first 3 songs with their Title and Chart Position

```python
print(df[['Title', 'Chart Position']].head(3))
```

✅ 3. Use `.loc[]` to display the first 4 songs and only their Artist and Length

```python
print(df.loc[0:3, ['Artist', 'Length']])
```

✅ 4. Display all songs where the Language is English and show only Title and Genre

```python
english_songs = df[df['Language'] == 'English']
print(english_songs[['Title', 'Genre']])
```

💡 Bonus: Songs released after 2015 and in English

```python
filtered = df[(df['Release Year'] > 2015) & (df['Language'] == 'English')]
print(filtered)
```

d. Filtering Rows

- **Based on conditions** (like SQL WHERE clause).
- Uses **boolean masks** to select rows.
- Best when you want rows that **meet specific criteria**.



Example:

```
df[df['Sales'] > 150]# Filter rows where Sales > 150
```

```
df[df['Product'] == 'Apples']# Filter rows where Product is Apples
```

Example Filtering Rows (Jupiter Notebook)

```
import pandas as pd

# Load your dataset
df = pd.read_excel("michael_jackson_songs.xlsx")

# 1. Filter songs released after 1985
print(df[df['Release Year'] > 1985])

# 2. Filter songs from the album 'Bad'
print(df[df['Album'] == 'Bad'])

# 3. Filter songs where Chart Position = 1
print(df[df['Chart Position'] == 1])
```

Google Colab file:
Filtering_Row.ipynb

Filtering Row.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text

```
import pandas as pd
import io

from google.colab import files
uploaded = files.upload()

# Get the actual filename from the 'uploaded' dictionary
file_name = list(uploaded.keys())[0]

# Load your dataset
df = pd.read_excel(io.BytesIO(uploaded[file_name]))  # Use the uploaded file content

# 1. Filter songs released after 1985
print(df[df['Release Year'] > 1985])

# 2. Filter songs from the album 'Bad'
print(df[df['Album'] == 'Bad'])

# 3. Filter songs where Chart Position = 1
print(df[df['Chart Position'] == 1])
```

```
Choose files Michael Jac...ng Data.xlsx
• Michael Jackson Song Data.xlsx(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) - 9650 bytes, last modified: 23/04/2025 - 100% done
Saving Michael Jackson Song Data.xlsx to Michael Jackson Song Data (1).xlsx
                    Title      Album  Release Year    Length     Genre  \
3           Smooth Criminal        Bad          1987  04:17:00       Pop
4            Black or White  Dangerous          1991  03:22:00  Pop Rock
5         Man in the Mirror        Bad          1988  05:19:00       Pop
6                       Bad        Bad          1987  04:07:00       Pop
8  The Way You Make Me Feel        Bad          1987  04:59:00       Pop
9             Heal the World  Dangerous          1991  06:25:00       Pop

   Chart Position
3               7
4               1
5               1
6               1
8               1
9               2
                    Title Album  Release Year    Length Genre  \
3           Smooth Criminal   Bad          1987  04:17:00   Pop
5         Man in the Mirror   Bad          1988  05:19:00   Pop
6                       Bad   Bad          1987  04:07:00   Pop
8  The Way You Make Me Feel   Bad          1987  04:59:00   Pop

   Chart Position
3               7
5               1
6               1
8               1
                         Title         Album  Release Year    Length  \
0                  Billie Jean      Thriller          1982  04:54:00
1                      Thriller      Thriller          1982  05:57:00
2                       Beat It      Thriller          1982  04:18:00
4               Black or White     Dangerous          1991  03:22:00
5            Man in the Mirror           Bad          1988  05:19:00
6                          Bad           Bad          1987  04:07:00
7  Don't Stop 'Til You Get Enough  Off the Wall          1979  06:05:00
8       The Way You Make Me Feel           Bad          1987  04:59:00

      Genre  Chart Position
0       Pop               1
1       Pop               1
2      Rock               1
4  Pop Rock               1
5       Pop               1
6       Pop               1
7     Disco               1
8       Pop               1
```

e. Sorting and Adding Columns

- **Sorting data**:
  - `df.sort_values(by='Price', ascending=False)`

- **Adding new columns**:
  - `df['Total'] = df['Price'] * df['Quantity']`

Example for Sorting

1. Sort the songs by **Release Year (oldest to newest)**.

```
df_sorted = df.sort_values(by='Release Year')
print(df_sorted)
```

2. Sort the songs by **Chart Position (best to worst)**.

```
df_sorted = df.sort_values(by='Chart Position')
print(df_sorted)
```

3. Sort the songs **alphabetically by Title**.

```
df_sorted = df.sort_values(by='Title')
print(df_sorted)
```

Example for Adding Columns

4. Add a new column called **"Decade"** based on the release year.

```
df['Decade'] = (df['Release Year'] // 10) * 10
print(df[['Title', 'Release Year', 'Decade']])
```

5. Add a column **"Hit"** with value **Yes** if Chart Position == 1, else **No**.

```
df['Hit'] = df['Chart Position'].apply(lambda x: 'Yes' if x == 1 else 'No')
print(df[['Title', 'Chart Position', 'Hit']])
```



CLASS ACTIVITY

Exploring Pop Songs Dataset using Pandas

Faculty of Artificial Intelligence

UTM UNIVERSITI TEKNOLOGI MALAYSIA

📦 Dataset: **pop_songs_dataset.xlsx**

- **Part A: Filtering Rows**

1. List all songs by the artist *Westlife*.
2. Find all songs that were released after the year 1999.
3. Display songs where the chart position was number 1.

- **Part B: Sorting Data**

4. Sort the dataset by *Release Year* in descending order.
5. Sort the songs alphabetically by *Title*.
6. Sort the songs by *Artist*, then by *Chart Position* (ascending).

- **Part C: Adding Columns**

7. Add a new column called *"Hit"* that shows "Yes" if the chart position is 1, otherwise "No".
8. Create a new column called *"Decade"* from the *Release Year*.
9. (Bonus) Add a column called *"Length (Seconds)"* by converting the "Length" (mm:ss) into total seconds.

f. Renaming and Dropping

- **Renaming columns**:
  - `df.rename(columns={'OldName': 'NewName'})`

- **Dropping columns or rows**:
  - `Column: df.drop('ColumnName', axis=1)`
  - `Row: df.drop(0, axis=0)`

Example for Renaming and Dropping

- Dataset: **renaming_and_dropping.xlsx**

1. Rename columns in DF

```
Album Name -> Album
Genre Type -> Genre
Release_Year -> Year
```

2. Dropping 1 columns (`Duration`)
3. Dropping multiple columns (`Duration and Genre Type`)
4. Dropping row (index 1)

Thriller	Thriller	1982	5:57	Pop

Google Colab file:
Renaming_and_dropping.ipynb

www.fai.utm.my

Introduction to Data Profiling

- **Definition:** Analyzing data to understand its structure, content, and quality.
- Objectives:
  i. Identify source, patterns, outliers, and anomalies
  ii. Assess data quality
  iii. Guide data cleansing efforts
- Tools: OpenRefine, Talend, Pandas Profiling

CLASS ACTIVITY

YouTube Duration: 5:06



According to YouTube video identify the process of data profiling.

https://www.youtube.com/watch?v=074G1EhcDu8

Data Profiling Techniques

a. Column analysis: null values, unique counts
b. Range and frequency analysis
c. Pattern recognition
d. Primary key analysis
e. Data type mismatches

a. Column Analysis

Purpose: Understand the basic characteristics of each column

Column Name	Example Values	Profiling Insights
email	null, a@b.com	5% missing values
gender	Male, Female, M	Inconsistent values, needs standardization
city	London, london	Case inconsistency, 15 unique cities

b. Range and Frequency Analysis

Purpose: Check value distribution and detect outliers.

Column	Min	Max	Common Values
age	0	112	25 (most frequent)
order_total	-50	10,000	Negative values are invalid

c. Pattern Recognition

Purpose: Identify expected vs. unexpected formatting.

Column	Example Value	Pattern Identified	Issue?
phone	(123) 456-7890	Matches U.S. phone format	No
phone	123-45-6789	Looks like SSN, not a phone #	Yes
email	user@domain.com	Valid email format	No
email	user@domain	Missing domain extension	Yes

d. Primary Key Analysis

Purpose: Ensure key uniqueness and completeness.

Example: Duplicates or missing values in a primary key violate integrity.

Column	Total Rows	Unique Values	Nulls	Issue?
customer_id	10,000	9,950	5	Duplicates + nulls

e. Data Type Mismatches

Purpose: Identify columns storing unexpected or inconsistent data types.

Example: price should be numeric, not a string with currency symbols.

Column	Expected Type	Sample Data	Issue?
date_of_birth	Date	12/03/1995	No
age	Integer	Twenty-five	Yes (string)
price	Numeric	USD 99.99	Yes (string mix)

Introduction to Data Cleansing

- Definition: Process of correcting or removing inaccurate, incomplete, or irrelevant data.
- Importance:
  i. Enhances decision-making
  ii. Improves operational efficiency
  iii. Ensures compliance with standards

Common Data Cleansing Methods

i. Handling missing data (deletion, imputation)
ii. Standardizing formats (dates, addresses)
iii. Removing duplicates
iv. Normalizing values (e.g., YES/No vs. Y/N)
v. Manual vs. automated cleaning

Feature	Manual Cleaning	Automated Cleaning
🧑💻 **How it works**	Human checks and edits data using tools like Excel	Software or scripts automatically identify and fix issues
⚙️ **Tools used**	Excel, Google Sheets, manual SQL queries	Python (Pandas), R, Talend, Informatica, OpenRefine
✅ **Best for**	Small datasets, nuanced judgment-based issues	Large datasets, repetitive or rule-based issues
⏳ **Speed**	Slow and time-consuming	Fast and scalable
🧠 **Expertise needed**	Basic data and domain knowledge	Technical skills (e.g., scripting, tool configuration)
🔁 **Reusability**	Not reusable—tasks must be repeated manually	Reusable scripts and workflows can be run repeatedly
🛑 **Risk of error**	Prone to human error, fatigue	Lower human error, but can introduce logic errors
🔍 **Transparency**	Very clear what changes are made	May need logging/auditing to track changes

Data Correction Strategies

- Manual correction with audit logs
- Automated correction rules (e.g., regex for email validation)
- Integration with master data sources
- Logging and tracking changes
- Role of business rules and data policies

Best Practices in Data Cleansing

- Always backup original data
- Start with profiling before cleansing
- Apply changes incrementally and test
- Maintain documentation of changes
- Implement validation checks after cleansing



CLASS ACTIVITY

Open the provided Excel file [**`Data cleaning for MRTQ2223_M5.xlsx`**] and answer the following questions after performing each cleaning step.

🔍 Part A: Data Profiling / Audit (Before Cleaning)

1.Missing Values

a. Which columns contain missing (null) values?
b. How many rows have at least one missing value?

2.Duplicate Records

a. How many duplicate rows are in the dataset?
b. Which method did you use to identify and remove duplicates?

3.Inconsistent Formats

a. Identify any inconsistencies in:
   i. Date of Birth format
   ii. Name casing
   iii. Address format and punctuation



CLASS ACTIVITY

Part B: Cleaning Actions

4.Handling Nulls

4. What approach did you use to fill or handle null values?
   (e.g., use “N/A”, fill with estimated value, drop row)

5.Standardizing Formats

4. What Excel functions did you use to clean the data?
   (e.g., PROPER, TRIM, TEXT, SUBSTITUTE)

6.Cleansing Decisions

4. Describe **one record** where you made a decision to change or standardize a value.
   (e.g., fixed email case, formatted date, unified address format)

g. Cleaning Data

When you load a dataset, some cells might be empty or contain NaN (Not a Number) — this means the data is **missing**.

a. Check for missing data

```
df.isnull()# Returns True for each cell that is missing.
df.isnull().sum() Counts how many missing values (nulls) are in each column.
```

b. Replace missing data

```
df.fillna(0)# Replace with 0
df['Genre Type'].fillna('Unknown', inplace=True)  #Replace with a specific value
df['Release_Year'].fillna(df['Release_Year'].mean(), inplace=True)
# Replace with the column mean (useful for numeric data)
```

Cleaning Data

c. Drop Missing data

```
df_clean = df.dropna()
```

Remove rows that contain any missing data

```
df_clean = df.dropna(how='all')
```

Drop only if **all values** in a row are missing

```
df_clean = df.dropna(subset=['Genre Type'])
```

Drop rows missing data **in a specific column**

Example

```
import pandas as pd

data = {
    "Name": ["Billie Jean", "Thriller", None, "Smooth Criminal"],
    "Year": [1982, 1982, None, 1987],
    "Genre": ["Pop", None, "Rock", None]
}

df = pd.DataFrame(data)

# Check missing values
print(df.isnull().sum())

# Replace missing genres with 'Unknown'
df['Genre'].fillna('Unknown', inplace=True)

# Drop rows where 'Name' is missing
df = df.dropna(subset=['Name'])

print(df)
```



CLASS ACTIVITY

Data Cleaning

📦 Dataset: `students_scores_dirty.xlsx`

🧠 **Questions to Answer:**

1. How many missing values are in each column before cleaning?
2. What are the mean, median, and mode used to fill the missing data?
3. How many rows had missing data originally?
4. What method did you use to fill missing data for each type of column?
5. Are there any remaining nulls after cleaning?

Tips:

1. **Import** the pandas library
2. **Load** the dataset using **pd.read_excel()**
3. **Check for missing data** using:
   - `df.isnull()`
   - `df.isnull().sum()`
4. **Replace missing data** using:
   - Mean/Median for numerical columns
   - Mode for categorical columns
5. **Drop rows** if necessary

Google Colab file:
Data_Cleaning.ipynb

www.fai.utm.my

h. Duplicates

a. Check for Duplicates

```
df.duplicated()
```
Returns True for each **duplicate row** (after the first occurrence).

```
df.duplicated().sum()
```
To count how many duplicates.

b. Remove Duplicate Rows

```
df_cleaned = df.drop_duplicates()
```
Removes all rows that are duplicates of previous ones.

c. Remove Duplicates Based on Specific Columns

```
df_cleaned = df.drop_duplicates(subset=['Title'])
```
Removing duplicates based on the `"Title"` column

Example

```
import pandas as pd

data = {
'Title': ['Thriller', 'Thriller', 'Beat It', 'Billie Jean', 'Beat It'],
'Year': [1982, 1982, 1982, 1982, 1982]
}

df = pd.DataFrame(data)

# Check duplicates
print(df.duplicated())

# Drop duplicate rows
df_unique = df.drop_duplicates()

print(df_unique)
```

Output

```
      Title        Year
0     Thriller     1982
2     Beat It      1982
3     Billie Jean  1982
```

www.fai.utm.my

CLASS ACTIVITY - Duplicates



i. Why is it important to check for duplicates in a dataset?

ii. Should we always remove duplicates? Why or why not?

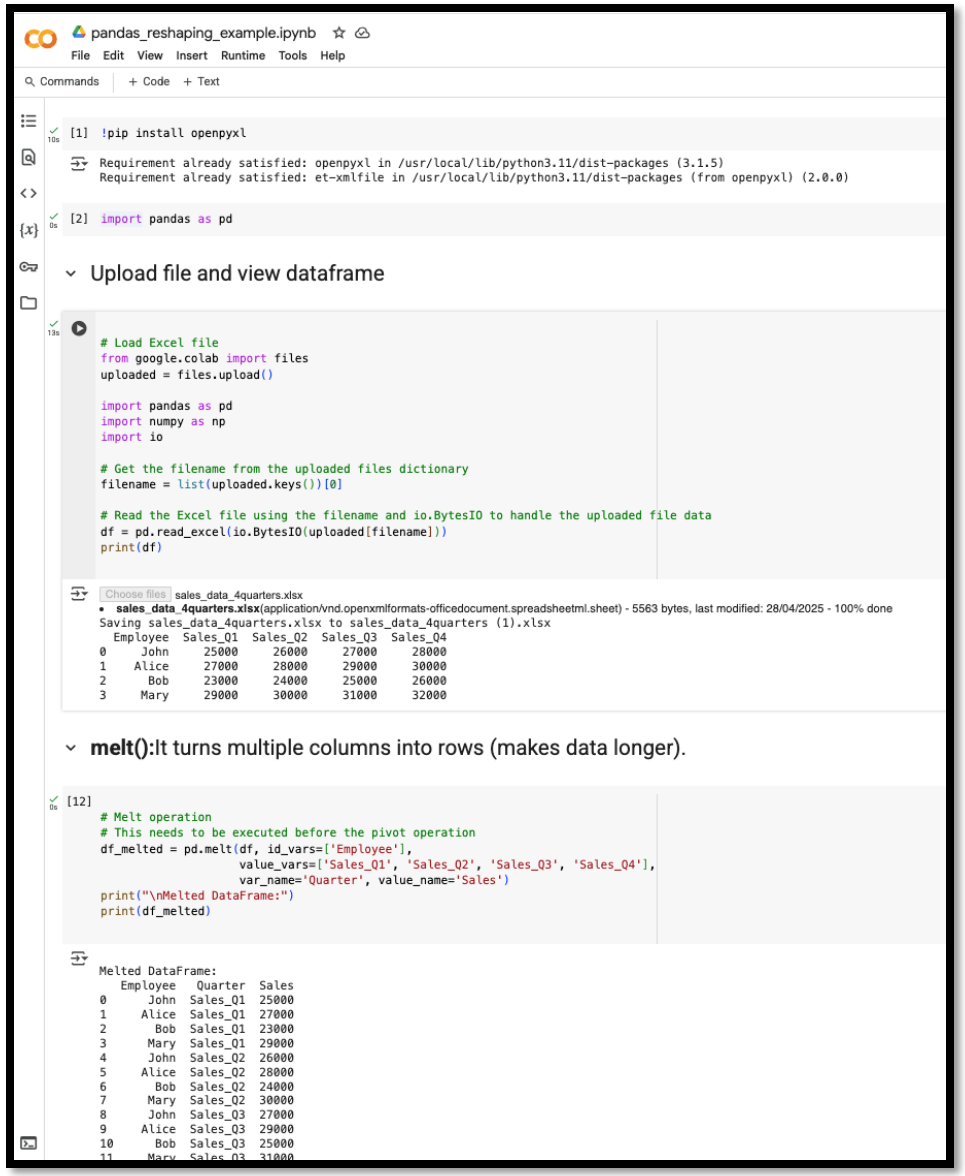
iii. What if only specific columns were duplicated? How could we check that?

i. Reshaping Data

Changing the structure or layout of a dataset — like rearranging rows and columns — without changing the data itself.

Common reshaping operations include:

- `melt()`: Opposite of pivot — it turns multiple columns into rows (makes data longer).
- `pivot()`: Turn unique values from one column into multiple columns.
- `stack()`: Compress columns into a new inner row index (makes data taller).
- `unstack()`: Opposite of stack — moves inner row index back to columns (makes data wider).
- `transpose()` `(.T)`: Flip rows and columns — rows become columns and columns become rows.
- `reshape()` (mostly using **NumPy** under the hood): Change the shape of arrays.



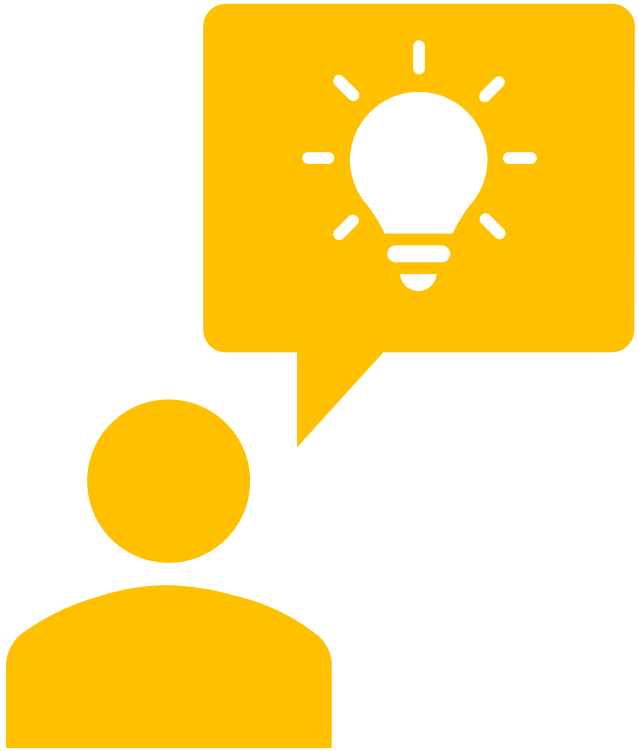
DataFrame: sales_data_4quarters.xlsx

	A	B	C	D	E
1	Employee	Sales_Q1	Sales_Q2	Sales_Q3	Sales_Q4
2	John	25000	26000	27000	28000
3	Alice	27000	28000	29000	30000
4	Bob	23000	24000	25000	26000
5	Mary	29000	30000	31000	32000

Google Colab file:

pandas_reshaping _example.ipynb

KNOWLEDGE CHECK



THANK YOU



Faculty of Artificial Intelligence (FAI)



http://fai.utm.my



Faculty of Artificial Intelligence, UTM



utmfai